# Lab#7 : Unit testing with Pytest (9 Tasks)

## How to Write & Run Unit Tests in PyCharm?

This applies to both unittest and pytest.

**A. Setting Up**

1. Open PyCharm → File → New Project
2. Create a folder structure like:

```
my_project/
  src/
    calculator.py
  tests/
    test_calculator.py
```

**B. Writing a Test File**

For unittest

```
import unittest
from src.calculator import add

class TestCalculator(unittest.TestCase):
  def test_add(self):
    self.assertEqual(add(2, 3), 5)

if __name__ == "__main__":
  unittest.main()
```

For pytest

```
from src.calculator import add

def test_add():
  assert add(2, 3) == 5
```

**C. Running Tests in PyCharm**

**Method 1 — Using Run Button**

1. Right-click the test file → Run 'test_calculator'
2. PyCharm will automatically detect:
   - pytest (if installed)
   - or fallback to unittest

**Method 2 — Run All Tests**

1. Go to Run → Edit Configurations

2. Click + → Python Tests → pytest (or unittest)
3. Select the test folder (tests/)
4. Click Run

**We will try to cover all major pytest concepts you studied:**

✓ markers

✓ skipping tests

✓ fixtures

✓ comparative/assert types

✓ parametrized testing

✓ parallel execution (pytest-xdist)

✓ scenario-driven tasks

Pytest **automatically discovers tests using naming conventions**.

✓ **Pytest test discovery rules:**

1. **Files** must be named:
   - test_*.py
   - OR *_test.py
2. **Functions** must start with:
   - test_
3. **Classes** must start with:
   - Test (and not have __init__)

**Example:**

✓ detected

test_math.py

test_login.py

user_test.py

✕ NOT detected

mathtests.py

loginTests.py

Your team is developing **EduTrack**, a cloud-based LMS used by schools and universities.

SQE engineers must ensure the reliability of core modules using **pytest**.

Your job is to create unit tests for major LMS components using best practices.

EduTrack modules involve:

1. **User Management** – login, roles, permissions
2. **Course Module** – create & list courses

3. **Grading Module** – calculate grades
4. **Analytics Module** – heavy computation (requires parallel tests)
5. **Database Layer** – uses fixtures
6. **Feature Flags** – may require skipping tests
7. **API Response Validator** – needs multiple assert types

## TASK SET 1 — Using Pytest Markers

Certain EduTrack tests take more time (e.g., verifying course searches). Some tests are "smoke", others "regression".

### Add markers to tests

Write tests for a function that returns course count.

```
def course_count(courses):
    return len(courses)
```

**Solution (test_course.py)**

```
import pytest
from src.course import course_count

@pytest.mark.smoke
def test_course_count_small():
    assert course_count(["Math", "Science"]) == 2

@pytest.mark.regression
def test_course_count_large():
    assert course_count(list(range(100))) == 100
```

### Running specific markers

```
pytest -m smoke
pytest -m "regression"
```

## TASK SET 2 — Skipping Tests

EduTrack has a feature flag:
**NEW_COURSE_FORMAT = False** – new format not yet released.

If the flag is false, skip related tests.

Skip tests based on feature flag

Code under test (course_format.py)

NEW_COURSE_FORMAT = False

```
def new_format_length(course):
    return len(course["name"])
```

**Solution (test_course_format.py)**

```
import pytest
from src.course_format import new_format_length, NEW_COURSE_FORMAT

@pytest.mark.skipif(not NEW_COURSE_FORMAT, reason="New course format not enabled")
def test_new_format_length():
    course = {"name": "Physics"}
    assert new_format_length(course) == 7
```

**TASK SET 3 — Fixtures**

EduTrack's DB layer needs a "fake database" during testing.
The fixture returns a mock DB containing users.

**Create fixture and use it in multiple tests**

**Code under test (user_db.py)**

```
def get_user_role(db, username):
    return db.get(username, "guest")
```

**Solution (conftest.py)**

```
import pytest

@pytest.fixture
def fake_db():
    return {
        "Sadia": "student",
```

```
    "admin": "administrator"
  }
```

**Solution (test_user_db.py)**

```
from src.user_db import get_user_role

def test_get_existing_role(fake_db):
  assert get_user_role(fake_db, "Sadia") == "student"

def test_get_unknown_role(fake_db):
  assert get_user_role(fake_db, "unknown") == "guest"
```

**TASK SET 4 — Parametrized Tests**

EduTrack's grading logic must support many grade input variations.

**Code (grade.py)**

```
def calculate_grade(score):
  if score < 0 or score > 100:
    raise ValueError("Invalid")
  if score >= 90:
    return "A"
  if score >= 80:
    return "B"
  return "C"
```

**Solution (test_grade.py)**

```
import pytest
from src.grade import calculate_grade

@pytest.mark.parametrize("score,grade", [
  (95, "A"),
  (80, "B"),
  (60, "C")
])
```

```
def test_grade(score, grade):
    assert calculate_grade(score) == grade
```

**TASK SET 5 — Comparative Assertions**

Analytics module returns performance reports; various assert types must be used.

**Code (analytics.py)**

```
def completion_rate(total, completed):
    return completed / total
```

**Solution (test_analytics.py)**

```
import pytest
from src.analytics import completion_rate

def test_completion_rate_value():
    assert completion_rate(10, 5) == pytest.approx(0.5)

def test_rate_type():
    result = completion_rate(20, 10)
    assert isinstance(result, float)

def test_rate_range():
    assert 0.0 <= completion_rate(10, 5) <= 1.0
```

**TASK SET 6 — Parallel Testing (pytest-xdist)**

Analytics module is expensive.

We run heavy tests in parallel.

**Install plugin**

```
pip install pytest-xdist
```

**Run in parallel with 4 CPUs**

```
pytest -n 4
```

**Heavy test example (test_heavy.py)**

```
import time
def test_heavy():
    time.sleep(2)
```

```
    assert True
```

Running with pytest -n 4 reduces execution time drastically.

**TASK SET 7 — Combining Fixtures + Parametrization + Marks**

Testing login under multiple roles.

**Code (auth.py)**

```
def login(user, db):
    if user not in db:
        return "invalid"
    return f"welcome_{db[user]}"
```

**Solution**

**conftest.py (reuse fixture)**

```
import pytest
@pytest.fixture
def user_roles():
    return {"Sadia":"student", "Natalia":"teacher"}
```

**test_auth.py**

```
import pytest
from src.auth import login

@pytest.mark.parametrize("user,expected", [
    ("Sadia", "welcome_student"),
    ("Natalia", "welcome_teacher"),
    ("unknown", "invalid")
])
@pytest.mark.regression
def test_login(user, expected, user_roles):
    assert login(user, user_roles) == expected
```

**TASK SET 8 — Skipping + Markers + Assertions**

A feature “auto grade scale” is not deployed yet.

**Code (auto_grade.py)**

```
AUTO_GRADE_ENABLED = False
def auto_grade(score):
    return score * 1.1
```

Solution

```
import pytest
from src.auto_grade import auto_grade, AUTO_GRADE_ENABLED


@pytest.mark.smoke
@pytest.mark.skipif(not AUTO_GRADE_ENABLED, reason="Auto-grade not active")
def test_auto_grade():
    assert auto_grade(100) == pytest.approx(110)
```

pytest.approx() is a **special comparison helper** used when:

- comparing floating-point numbers
- dealing with calculations that may have rounding errors
- verifying results that should be approximately equal, not exactly equal

**TASK 9— Add Tests Using Collection & Comparative Assertions**

FinPay has added a small analytics module:

**src/analytics.py**

```
def top_transactions(transactions, limit=3):
    """Return the top 'limit' largest transactions in descending order."""
    return sorted(transactions, reverse=True)[:limit]


def classify_amount(amount):
    if amount < 50:
        return "small"
    if amount < 200:
        return "medium"
    return "large"
```

You must add tests that:

- validate collection-based results
- check ordering
- check membership
- use comparative asserts (>, <, <=, etc.)
- test classification boundaries

**test_analytics_regression.py**

```
import pytest
from src.analytics import top_transactions, classify_amount


@pytest.mark.regression
def test_top_transactions_collection():
    transactions = [120, 500, 80, 200, 999, 10]

    result = top_transactions(transactions)

    # Collection asserts
    assert isinstance(result, list)
    assert len(result) == 3

    # Values in the result (unordered membership)
    assert 999 in result
    assert 500 in result
    assert 200 in result

    # Exact order expected (ordered assert)
    assert result == [999, 500, 200]



@pytest.mark.regression
def test_top_transactions_custom_limit():
    transactions = [5, 20, 100, 60]
    result = top_transactions(transactions, limit=2)

    # Check only top 2 elements returned
    assert result == [100, 60]



@pytest.mark.regression
def test_classify_amount_small():
```

```
    assert classify_amount(10) == "small"
    assert classify_amount(49) == "small"
    # comparative assert
    assert 49 < 50


@pytest.mark.regression
def test_classify_amount_medium():
    assert classify_amount(50) == "medium"
    assert classify_amount(199) == "medium"
    # comparative assert
    assert 50 >= 50
    assert 199 < 200


@pytest.mark.regression
def test_classify_amount_large():
    assert classify_amount(200) == "large"
    assert classify_amount(500) == "large"
    # comparative asserts
    assert 200 >= 200
    assert 500 > 200
```